



Fórum Respostas

Documentação de Arquitetura do Projeto

Versão 1.0 — Julho 2026

Stack: HTML/CSS/JS · Node.js · Express · PostgreSQL 16 ·

Docker

www.projetosdinamicos.com.br

Índice

1. Visão Geral	4
2. Estrutura do Projeto	5
2.1 Diretórios Versionados	5
2.2 Artefatos Gerados (install_forum.sh)	5
3. Stack Tecnológica	7
4. Frontend	8
4.1 Páginas	8
4.2 Componentes Compartilhados	9
4.3 api.js — Cliente HTTP	9
4.4 Folha de Estilo	10
5. Backend (API Express)	11
5.1 Middleware	11
5.2 Rotas da API	12
6. Banco de Dados	15
6.1 Schema — 7 Tabelas	15
6.2 Diagrama Entidade-Relacionamento	16
7. Fluxos de Negócio	18

7.1 Autenticação	18
7.2 Melhor Resposta (Moderação)	18
7.3 Votação (Toggle)	19
7.4 Destaques por Categoria	19
8. Implantação	21
8.1 Docker Compose	21
8.2 Nginx Reverse Proxy	21
8.3 Script install_forum.sh	22

1. Visão Geral

O **Fórum Respostas** é uma plataforma colaborativa de perguntas e respostas inspirada no Yahoo! Respostas. Usuários podem criar perguntas em categorias temáticas, responder, votar em respostas e comentar. O autor da pergunta tem o poder de eleger a **melhor resposta**, que destaca a pergunta na página inicial e concede reputação ao autor da resposta.

 **Arquitetura resumida:** Frontend estático (HTML+CSS+JS) consome uma API REST Node.js/Express que persiste em PostgreSQL 16. Tudo roda em containers Docker orquestrados por Docker Compose, com Nginx como proxy reverso.

HTML5**JavaScript (Vanilla)****Node.js 20****PostgreSQL 16****Docker****Nginx**

2. Estrutura do Projeto

2.1 Diretórios Versionados (repositório)

```
forum/
├── .gitignore
├── install_forum.sh      # Script de instalação (1065 linhas)
├── index.html           # Página inicial (hero + stats +
destaques + categorias)
├── perguntas.html       # Lista de perguntas com filtros e
paginação
├── pergunta.html        # Detalhe da pergunta + respostas +
comentários
├── nova-pergunta.html    # Formulário de nova pergunta
├── categorias.html       # Grade de categorias
├── cadastro.html         # Cadastro de usuário
├── login.html            # Login
├── perfil.html           # Perfil do usuário
├── minhas-perguntas.html # Perguntas do usuário logado
├── termos.html           # Termos de uso
├── privacidade.html      # Política de privacidade
├── admin/
│   └── index.html        # CRUD administrativo
├── components/
│   ├── header.js         # Header com search + user menu
│   └── footer.js         # Footer institucional
├── static/
│   ├── css/default.css   # Estilos globais (892 linhas)
│   ├── js/api.js         # Cliente HTTP + helpers
│   └── img/avatar-default.svg
└── templates/
    └── docker-compose.yml # Template de orquestração
```

2.2 Artefatos Gerados em Tempo de Instalação

O script `install_forum.sh` gera dinamicamente em `/var/www/forum/` :

```

/var/www/forum/
├─ docker-compose.yml      # Orquestração final (com variáveis
expandidas)
├─ .env                    # Variáveis de ambiente (token, senhas,
URLs)
├─ api/
│   ├─ Dockerfile          # Node 20-alpine
│   ├─ package.json        # express, pg, cors, dotenv
│   └─ src/server.js       # Servidor Express (~740 linhas)
├─ db/
│   └─ init/
│       ├─ 01-schema.sql   # DDL das 7 tabelas
│       └─ 02-seed.sh      # Admin + 10 categorias + settings
├─ pgdata/                # Volume persistente do PostgreSQL
├─ uploads/               # Uploads de avatar/imagens
├─ backups/               # Backup automático
└─ static/js/api_token.js  # Token público do cliente

```

⚡ **Design híbrido:** O repositório contém apenas o "projeto fonte". A cada `install` ou `reset`, todo o backend é regenerado com credenciais e token únicos, garantindo instalações isoladas e reproduzíveis.

3. Stack Tecnológica

Frontend

HTML5 semântico · CSS3 (Grid, Flexbox, Custom Properties, Animations) · JavaScript Vanilla (ES5/ES6+) · Font Awesome 6 · Google Fonts (Inter)

Backend

Node.js 20 · Express 4 · pg (node-postgres) · cors · dotenv · Auto-criação de tabelas e colunas via `information_schema`

Banco

PostgreSQL 16 Alpine · 7 tabelas · UNIQUE constraints · Chaves estrangeiras · ON DELETE CASCADE · TIMESTAMP automáticos

Infraestrutura

Docker Compose · 2 serviços (db + api) · Healthcheck · Volume persistente · Nginx reverse proxy · API via subpath (/forum/)

4. Frontend

4.1 Páginas

O frontend é composto por **11 páginas HTML estáticas** mais uma página de administração. Cada página carrega componentes de header e footer via JavaScript, e consome a API através do cliente `api.js`.

`index.html`

Hero com CTA · Stats (perguntas, respostas, usuários, resolvidas) · Destaques por categoria
· Grade de categorias · Perguntas recentes

`perguntas.html`

Lista com filtros (categoria, status, busca textual) · Paginação · Query params na URL

`pergunta.html`

Detalhe da pergunta · Respostas ordenadas (melhor primeiro) · Votação up/down ·
Comentários · Ações do autor (fechar, escolher melhor resposta)

`nova-pergunta.html`

Formulário com título, descrição, categoria (select), tags · Validação client-side

`login.html / cadastro.html`

Autenticação · Token armazenado em localStorage com expiry de 24h · Redirect após login

`perfil.html`

Avatar, bio, reputação · Edição de perfil · Lista de perguntas do usuário

`categorias.html`

Grade de todas as categorias com contagem de perguntas

 admin/index.html

CRUD genérico sobre qualquer tabela (categorias, perguntas, respostas, login) · Apenas admin · Tabela dinâmica com actions

4.2 Componentes Compartilhados

Cada página inclui os componentes via `innerHTML` com script tag (técnica síncrona que funciona em todos os browsers):

```
<div id="header-placeholder"></div>
<script>
  document.getElementById('header-placeholder').innerHTML =
    '<script src="components/header.js"></script>';
</script>
```

 header.js

Header fixo (sticky) · Logo + search com autocomplete · Nav links · User menu com dropdown (avatar, nome, perfil, sair) · Botões de auth condicionais · **Barra de progresso** nas requisições

 footer.js

Footer escuro com 4 colunas (marca, navegação, conta, legal) · Links institucionais

4.3 api.js — Cliente HTTP

O arquivo `static/js/api.js` é o coração da comunicação com o backend. Expõe no `window` todas as funções necessárias:

 Autenticação

`getToken()` · `getUsuario()` · `rmToken()` — lê/remove do localStorage, com verificação de expiry (24h)

Requisições

`go(method, endpoint, data, cb)` — XHR wrapper com token Bearer, auto-parse JSON, tratamento 401, **barra de progresso**

Helpers

`esc(s)` (escape HTML) · `fmtDate(d)` (relativo: "agora", "5min", "3d", data pt-BR) · `imgUrl(url)` · `notificar(msg, tipo)`

Page Enter

`pageEnter()` adiciona classe fade-in no `.main-content` via `DOMContentLoaded`

4.4 Folha de Estilo

Um único arquivo `default.css` (~892 linhas) cobre todo o sistema visual:

Design System

Variáveis CSS (cores, sombras, raios) · Fonte Inter via Google Fonts · Sistema de grid (container 1200px) · Token de espaçamento consistente

Componentes

Header · Cards (pergunta, categoria, destaque) · Botões (primary, outline, accent, ghost) · Formulários · Paginação · Tabela admin

Animações

`pageFadeIn` (fade + translateY) · `shimmer` (skeleton loading) · `spin` (spinner) · `slideIn` (notificação) · Suporte a `prefers-reduced-motion`

Responsivo

Breakpoint 768px (header, hero, grids, formulários) · Breakpoint 900px (sidebar) · Layouts adaptativos com auto-fill/minmax

5. Backend (API Express)

Servidor Node.js gerado dinamicamente em `api/src/server.js` (~740 linhas). Conecta ao PostgreSQL via `pg.Pool` e expõe uma API RESTful.

5.1 Middleware

1 JSON Parser

`express.json({ limit: '50mb' })` + `urlencoded` — suporta payloads grandes (uploads)

2 CORS

Origins configuráveis via `SITE_URL` + domínios fixos (`projetosdynamics.com.br`, `api.*`). Métodos: GET, POST, PUT, DELETE, OPTIONS

3 Autenticação Bearer

Valida `Authorization: Bearer <token>` em todas as rotas exceto GET, HEAD, `/health` e `/auth/*`. Token único comparado com `API_TOKEN` da env

4 Auto-migration

`tabelaExiste()` + `garantirTabela()` + `garantirColunas()` — cria tabelas e colunas dinamicamente via `information_schema`. Permite schema flexível sem migrations

5.2 Rotas da API

Método	Rota	Descrição
GET	/health	Healthcheck com status do banco
POST	/auth/login	Login por email+senha → token + usuário
POST	/auth/cadastro	Registro (nome, email, senha) → token + usuário
GET	/auth/usuario	Dados do usuário por ID (query)
PUT	/auth/usuario	Atualizar nome, avatar, bio
GET	/categorias	Listar categorias com total de perguntas
GET	/perguntas	Listar perguntas (filtros: categoria, status, busca, paginação)
GET	/perguntas/:id	Detalhe da pergunta (autor, categoria) + incrementa view
POST	/perguntas	Criar pergunta
PUT	/perguntas/:id	Atualizar pergunta (fechar, editar)
DELETE	/perguntas/:id	Excluir pergunta + respostas em cascata
GET	/respostas/pergunta/:id	Listar respostas (melhor primeiro, votos DESC)
POST	/respostas	Criar resposta
PUT	/respostas/:id/melhor-resposta	Marcar melhor resposta (autor da pergunta apenas)
POST	/respostas/:id/votar	Votar up/down (toggle se mesmo tipo)
GET	/votos/resposta/:id	Ver votos de uma resposta
GET	/comentarios/resposta/:id	Listar comentários de uma resposta
POST	/comentarios	Criar comentário

Método	Rota	Descrição
GET	/dashboard	Dashboard completo (stats, destaques, top usuários)
GET	/search?q=	Busca textual em perguntas (LIKE %term%)
GET	/:tabela	CRUD genérico — listar registros (admin)
POST	/:tabela	CRUD genérico — criar registro (admin)
PUT	/:tabela/:id	CRUD genérico — atualizar registro (admin)
DELETE	/:tabela/:id	CRUD genérico — excluir registro (admin)

6. Banco de Dados

6.1 Schema — 7 Tabelas

settings

chave PK · valor — Configurações chave-valor (site_nome, admin_email)

login

id PK · nome · email UNIQUE · senha · admin · avatar · bio · reputacao ·
created_at

categorias

id PK · nome · descricao · icone · slug UNIQUE · created_at

perguntas

id PK · titulo · descricao · categoria_id FK · usuario_id FK · status ·
tags · visualizacoes · created_at · atualizado_em

respostas

id PK · pergunta_id FK CASCADE · usuario_id FK · conteudo ·
melhor_resposta · votos · created_at

comentarios

id PK · resposta_id FK CASCADE · usuario_id FK · conteudo · created_at

votos

id PK · usuario_id FK · resposta_id FK CASCADE · tipo ·
UNIQUE(usuario_id, resposta_id)

6.2 Diagrama Entidade-Relacionamento

login**PK** idnome, email **UNIQUE**, senha, admin, avatar, bio, reputacao, created_at

| 1 | ← || N |

perguntas**PK** id

titulo, descricao, tags, status, visualizacoes

FK usuario_id → login.id**FK** categoria_id → categorias.id

| 1 | ← || N |

respostas**PK** id

conteudo, melhor_resposta, votos

FK pergunta_id → perguntas.id **CASCADE****FK** usuario_id → login.id

| 1 | ← || N |

comentarios**PK** id

conteudo

FK resposta_id → respostas.id **CASCADE****FK** usuario_id → login.id**votos****PK** id

tipo (up/down)

FK usuario_id → login.id**FK** resposta_id → respostas.id **CASCADE****UNIQUE**(usuario_id, resposta_id)**categorias****PK** idnome, descricao, icone, slug **UNIQUE****settings****PK** chave

valor

7. Fluxos de Negócio

7.1 Autenticação

1 Login / Cadastro

Usuário envia email+senha para `/auth/login` ou `/auth/cadastro`. Servidor valida no banco e retorna `{ token, usuario, session_expiry }`

2 Armazenamento local

O frontend salva em `localStorage` com prefixo do `APP_NAME` ("forum_token", "forum_usuario", "forum_session_expiry")

3 Requisições autenticadas

`api.js` anexa `Authorization: Bearer <token>` em toda requisição. Se o servidor retornar 401, o token é removido automaticamente

4 Expiração

`session_expiry` guarda `Date.now() + 86400000` (24h). `getToken()` verifica se expirou e limpa o storage automaticamente

5 Admin

Usuários com flag `admin = true` têm acesso à página `/admin/`. O frontend redireciona se `usuario.admin` for falso

7.2 Melhor Resposta (Moderação)

1 Autor escolhe

Na página da pergunta, se o usuário logado é o autor e a pergunta está "aberta", um botão "Melhor Resposta" aparece em cada resposta

2 Validação no servidor

`PUT /respostas/:id/melhor-resposta` verifica se `req.body.usuario_id` é o dono da pergunta. Caso contrário, retorna 403

3 Efeitos em cascata

Ao marcar: (a) `melhor_resposta = false` em todas as respostas da pergunta; (b) `melhor_resposta = true` na escolhida; (c) status da pergunta → "resolvida"; (d) reputação do autor da resposta += 10

4 Destaque na Home

O dashboard retorna uma query `DISTINCT ON (categoria_id)` que seleciona uma pergunta resolvida por categoria (a com mais votos). Essas aparecem como "Destaques" na página inicial

7.3 Votação (Toggle)

1 Voto único por usuário

A constraint `UNIQUE(usuario_id, resposta_id)` na tabela `votos` garante que cada usuário vote apenas uma vez por resposta

2 Toggle inteligente

`POST /respostas/:id/votar` implementa três cenários: (a) mesmo tipo → remove voto (downvote); (b) tipo diferente → troca voto (delta = 2); (c) novo voto → insere (delta = ±1). O campo `respostas.votos` é atualizado atomicamente

7.4 Destaques por Categoria

O endpoint `GET /dashboard` executa uma consulta que:

```
SELECT DISTINCT ON (p.categoria_id)
  p.id, p.titulo, p.categoria_id, p.created_at,
  c.nome as categoria_nome, c.icone as categoria_icone,
  l.nome as autor_nome, l.avatar as autor_avatar,
  r.id as resposta_id, r.conteudo as resposta_conteudo,
  la.nome as resposta_autor_nome, la.avatar as resposta_autor_avatar
FROM perguntas p
INNER JOIN respostas r ON r.pergunta_id = p.id AND r.melhor_resposta =
true
LEFT JOIN categorias c ON c.id = p.categoria_id
LEFT JOIN login l ON l.id = p.usuario_id
LEFT JOIN login la ON la.id = r.usuario_id
WHERE p.status = 'resolvida'
ORDER BY p.categoria_id, r.votos DESC
```



Lógica: Para cada categoria, uma pergunta resolvida com melhor resposta aparece em destaque. Se houver múltiplas, a com mais votos vence. Isso incentiva respostas de qualidade e mantém a página inicial sempre atualizada com conteúdo relevante.

8. Implantação

8.1 Docker Compose

Dois serviços definidos no `docker-compose.yml` gerado:

```
services:
  db:
    image: postgres:16-alpine
    container_name: forum-db
    # healthcheck com pg_isready
    volumes:
      - ./db/init/01-schema.sql:/docker-entrypoint-initdb.d/
      - ./db/init/02-seed.sh:/docker-entrypoint-initdb.d/
      - ./pgdata:/var/lib/postgresql/data

  api:
    build: ./api
    container_name: forum-api
    depends_on:
      db: { condition: service_healthy }
    environment:
      DB_HOST: db
      API_TOKEN: "${API_TOKEN}"
    ports:
      - "3005:3000"
```

8.2 Nginx Reverse Proxy

A API é exposta publicamente via subpath `/forum/` no domínio

`api.projetosdynamics.com.br`. O script cria um snippet `/etc/nginx/forum-locations.conf`:

```
location /forum/ {
    rewrite ^/forum/(.*) /$1 break;
    proxy_pass http://127.0.0.1:3005/;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

8.3 Script install_forum.sh

O script centraliza todo o ciclo de vida do projeto:

install

Cria diretórios, gera todos os artefatos (server.js, schema, seed, Dockerfile, .env, api_token.js), constrói containers com `docker compose up -d --build`, configura Nginx

uninstall

Derruba containers, remove `/var/www/forum/` e o snippet do Nginx

logs

Exibe as últimas 30 linhas do container `forum-api`

reset

Remove volumes (`-v`) e dados (`pgdata/`), executa install novamente do zero

 **Segurança:** A cada instalação um `API_TOKEN` de 64 caracteres hex é gerado via `openssl rand -hex 32`. O token é injetado no `.env` do servidor e no `static/js/api_token.js` do cliente. O repositório nunca armazena tokens ou senhas reais.

 **Uso:** `sudo bash install_forum.sh install` — e o fórum está no ar em segundos. Acesse a API em <https://api.projetosdynamics.com.br/forum/> e o frontend em <https://www.projetosdynamics.com.br/forum/> .